Лабораторная работа №1

Симметричная криптография. ГОСТ 28147-89

Пояснительная записка к защите. Реализация: internal/crypto/gost, язык Go

---

Содержание

1. Что сделано в работе 2
2. Обозначения 2
3. Паспорт алгоритма 2
   3.1. Почему стандарт не задаёт узлы замены 3
4. Устройство шифра 3
   4.1. Сеть Фейстеля 3
   4.2. Функция раунда 3
   4.3. Расписание ключей 4
   4.4. Один цикл целиком 4
5. Режимы работы 4
   5.1. Простая замена, *ECB* 4
   5.2. Гаммирование 5
   5.3. Гаммирование с обратной связью 5
   5.4. Имитовставка 5
   5.5. Сводная таблица режимов 6
6. Расхождения со стандартом и недочёты 6
7. Реализация 7
   7.1. Структура проекта 7
   7.2. Формат шифртекста 7
   7.3. Методы 8
8. Проверка работоспособности 8
9. Вопросы на защите 8

1. Что сделано в работе

Реализован блочный шифр ГОСТ 28147-89 и четыре режима его применения: простая замена, гаммирование, гаммирование с обратной связью и выработка имитовставки. Это в точности объём задания.

Пункт	Что	Функции Go	Проверка
п. 2.1	Блок 64 бита, 32 цикла, функция раунда, расписание ключей	`BlockEncrypt32` / `BlockDecrypt32`, `BlockEncryptParts` / `BlockDecryptParts`	тест против уравнений п. 2.1, круговой тест блока
п. 2.2	Расписание ключей для расшифрования	`KeysForRoundDecrypt`	тест порядка подключей
п. 3.1	Режим гаммирования	`Gamma`, `GostGammaEncrypt` / `GostGammaDecrypt`	тест констант приращения
п. 4.1	Режим гаммирования с обратной связью	`GammaWithFeedbackEncrypt` / `GammaWithFeedbackDecrypt`	разбор кода, см. раздел о расхождениях
п. 5.3	Выработка имитовставки	`MACProduce`, `GammaEncryptWithMAC` / `GammaDecryptWithMAC`	тест против формулы п. 5.3 для четырёх длин

> **Ключевые числа для защиты**
> Блок – **64 бита**. Ключ – **256 бит**. Циклов – **32**. Подключ – **32 бита**. Узлов замены – **восемь**, каждый переводит 4 бита в 4 бита. Имитовставка – первые 16 циклов, длина от 1 до 32 бит.

2. Обозначения

Запись	Смысл
a, b	Две половины блока по 32 бита, накопители N_1 и N_2; в коде это `left` и `right`.
$f(L, K)$	Функция раунда: сложение с подключом, замена через узлы, сдвиг на 11 бит.
$K_1, \ldots, K_8$	Восемь 32-битных слов ключа (в стандарте – накопители $X_0, \ldots, X_7$).
k_i	Подключ i-го цикла, выбранный из $K_1, \ldots, K_8$ по расписанию.
π_j	Узел замены j: перестановка чисел от 0 до 15. Восемь узлов вместе – блок подстановки.
S	Синхропосылка, 64 бита. Не секретная, передаётся вместе с шифртекстом.
Γ_i	64-битный блок гаммы, складываемый с i-м блоком данных по модулю 2.
N_3, N_4	Счётчики гаммы: N_3 растёт по модулю 2^{32}, N_4 – по модулю $2^{32}-1$.
C_1, C_2	Константы приращения счётчиков: $C_1 =$ `0x01010104`, $C_2 =$ `0x01010101`.
I_l	Имитовставка длиной l бит, $1 \le l \le 32$.

3. Паспорт алгоритма

Параметр	Значение
Год, статус	1989, действующий в РФ как ГОСТ Р 28147-89; основа ГОСТ Р 34.12-2015 «Магма»
Тип	симметричный блочный шифр, **сеть Фейстеля**
Размер блока	**64 бита** (8 байт)
Размер ключа	**256 бит** (32 байта)

Число циклов	32 при шифровании и расшифровании, 16 при выработке имитовставки
Размер подключа	32 бита
Нелинейность	блок подстановки: восемь узлов 4 бит в 4 бит
Порядок байтов	little-endian во всех словах

3.1. Почему стандарт не задаёт узлы замены

Блок подстановки в ГОСТ 28147-89 объявлен **долговременным ключевым элементом**: стандарт требует только, чтобы каждый из восьми узлов был перестановкой чисел от 0 до 15, а конкретное наполнение поставляется отдельно. В работе взят набор `pi_1, dots, pi_8` из параметра `id-tc26-gost-28147-param-Z` (RFC 7836).

Практическое следствие: **«правильный ГОСТ» не один**. Две реализации с разными узлами замены дают разные шифртексты на одном ключе и одном блоке, и обе соответствуют стандарту. Это частый вопрос, и правильный ответ именно такой.

4. Устройство шифра

4.1. Сеть Фейстеля

Блок делится на две половины a и b. Один цикл:

$$a_j = f(a_{j-1}, k_j) \oplus b_{j-1}$$
$$b_j = a_{j-1}$$

Половины не меняются местами явно – они меняются ровно тем, что старое a переписывается в b, а в a попадает XOR старого b с результатом f. Отсюда обратимость при любой функции f: чтобы отменить цикл, достаточно вернуть a на место и повторить f.

> **32-й цикл — исключение**
> По тексту стандарта в последнем, 32-м цикле результат из сумматора пишется в N_2, а N_1 **сохраняет старое заполнение**: роли накопителей не меняются местами. В коде это не выделено специально – цикл с обменом половин, повторённый 32 раза, математически даёт ровно то же, поэтому отдельная ветка для последнего цикла не нужна. Но при «наивной» реализации по уравнениям этот момент забывают чаще всего.

4.2. Функция раунда

Для входа L и подключа K:

1. $S = (L + K) \bmod 2^{32}$;
2. S разбивается на восемь 4-битных частей $s_0, \ldots, s_7$, начиная с младшей;
3. каждая часть заменяется: $s_{j'} = \pi_j(s_j)$;
4. части склеиваются в 32-битное слово;
5. выполняется циклический сдвиг влево на 11 бит.

```
func f(left uint32, subkey uint32) uint32 {
  s := left + subkey

  var result uint32
  for i := range 8 {
    x := uint8((s >> (4 * i)) & 0xF)
    result |= uint32(sbox[i][x]) << (4 * i)
  }

  return bits.RotateLeft32(result, 11)
}
```

Сложение для `uint32` в Go уже идёт по модулю 2^{32}.

4.3. Расписание ключей

Ключ – восемь слов $K_1, \ldots, K_8$ младшим байтом вперёд. При шифровании подключи идут трижды прямо $(K_1, \ldots, K_8)$, затем раз в обратном порядке $(K_8, \ldots, K_1)$:

$$X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7, \quad X_0, \ldots, X_7, \quad X_7, \ldots, X_0$$

При расшифровании первые восемь идут прямо, остальные 24 – циклически в обратном порядке. Для имитовставки берутся первые 16 циклов, то есть $K_1, \ldots, K_8$ дважды, без разворота.

```
func KeysForRoundEncrypt(key []byte, rounds int) []uint32 {
  // ...
  const reversedFrom = 24

  for i := range rounds {
    if i < reversedFrom {
      result[i] = subkeys[i%8]
    } else {
      result[i] = subkeys[7-i%8]
    }
  }

  return result
}
```

4.4. Один цикл целиком

```
func BlockEncryptParts(input []byte, key []byte, rounds int) (uint32, uint32) {
  left := binary.LittleEndian.Uint32(input[0:4])
  right := binary.LittleEndian.Uint32(input[4:8])

  subkeys := KeysForRoundEncrypt(key, rounds)

  for i := range rounds {
    left, right = right^f(left, subkeys[i]), left
  }

  return right, left
}
```

Одна функция и для шифрования, и для расшифрования – различаются только расписание ключей и порядок половин. `rounds` передаётся параметром, потому что для имитовставки нужно 16, а не 32.

5. Режимы работы

5.1. Простая замена, *ECB*

$$Y_i = F(X_i), \quad X_i = F^{-1}(Y_i)$$

В коде данные дополняются по *PKCS#7* до кратности 8 байт, каждый блок шифруется независимо, при расшифровании дополнение снимается.

Свойства. Длина шифртекста кратна 8 и больше длины открытого текста. Синхропосылки нет, одинаковые блоки дают одинаковый шифртекст. Зашифрование и расшифрование параллелятся, есть произвольный доступ к блокам. Ошибка в бите шифртекста портит весь соответствующий блок открытого текста.

> **Про PKCS#7 в простой замене**
> Стандарт определяет простую замену для полных блоков. Дополнение – решение реализации, а не

требование стандарта. В отличие от `belt` из работы №2, где применяется кража шифртекста и длина сохраняется точно, здесь длина шифртекста всегда кратна 8.

5.2. Гаммирование

Синхропосылка шифруется, и полученные два слова становятся счётчиками. Далее на каждом шаге счётчики увеличиваются на константы и склеенный блок шифруется:

$$n_3 \leftarrow (n_3 + C_2) \bmod 2^{32}, \quad n_4 \leftarrow (n_4 + C_1) \bmod (2^{32} - 1)$$
$$\Gamma_i = F(n_3 \parallel n_4)$$
$$Y_i = X_i \oplus \Gamma_i, \quad X_i = Y_i \oplus \Gamma_i$$

Константы из приложения 2 стандарта: $C_1 =$ `0x01010104`, $C_2 =$ `0x01010101`.

n_4 растёт по модулю $2^{32} - 1$, а не по модулю 2^{32}: при переносе из `uint32` счётчик дополнительно увеличивается на единицу, а значение `0xFFFFFFFF` трактуется как ноль.

Свойства. Дополнения нет, длина сохраняется. Расшифрование устроено так же, поэтому обратное преобразование F^{-1} не используется. Гамма не зависит от данных, значит она может быть вычислена заранее и параллельно, поддерживается произвольный доступ к блокам. Ошибка в бите шифртекста портит ровно тот же бит открытого текста и не размножается.

5.3. Гаммирование с обратной связью

Регистр обратной связи – предыдущий блок **шифртекста**. В коде для каждого блока вырабатывается восемь байт гаммы из этого блока и выполняется XOR:

```
prev := iv

for i := 0; i < len(padded); i += 8 {
  gamma := Gamma(prev, key, 8)

  block := padded[i : i+8]
  for j := range block {
    cipherText[i+j] = block[j] ^ gamma[j]
  }

  prev = cipherText[i : i+8]
}
```

Свойства. Дополнение по *PKCS#7*, длина кратна 8. Зашифрование строго последовательное. Ошибка в бите Y_i портит тот же бит X_i и **весь** блок X_{i+1}, после чего поток восстанавливается – самосинхронизация через один блок.

5.4. Имитовставка

$$N_1 = N_2 = 0, \quad (N_1, N_2) = F^{16}_{\text{блоки с XOR}}$$

Последовательно для каждого блока: содержимое блока складывается с N_1 и N_2 поразрядно по модулю 2, затем пара шифруется 16 циклами, и результат переписывается в N_1, N_2. Имитовставка – первые l бит N_1:

$$I_l = [a_{32-l+1} \parallel \ldots \parallel a_{32}]$$

```
n1, n2 := parts(padded, key, 16)

for i := 8; i < len(padded); i += 8 {
  n1 ^= binary.LittleEndian.Uint32(padded[i : i+4])
  n2 ^= binary.LittleEndian.Uint32(padded[i+4 : i+8])
  // ...
  n1, n2 = parts(combined, key, 16)
}
```

```
binary.BigEndian.PutUint32(result, n1)
return result[:byteLen]
```

Последний блок дополняется **нулями**, а не значением длины – это принципиальное отличие от дополнения в простой замене. `BigEndian.PutUint32` в последней строке не случаен: так старшие разряды N_1 попадают в начало результата, то есть выбираются именно ведущие биты, как требует стандарт.

Свойства. Целостность и подлинность: имитовставка вычисляется на ключе, и получатель может убедиться, что сообщение не меняли. Вероятность угадать усечённую до l бит имитовставку – 2^{-l}. При шифровании с имитовставкой сначала шифруется сообщение, имитовставка считается по **открытому** тексту, поэтому проверка идёт после расшифрования.

5.5. Сводная таблица режимов

Свойство	ECB	Гамма	Обр. связь	MAC
Использует F^{-1}	да	нет	нет	нет
Нужна синхропосылка	нет	да	да	нет
Дополнение	PKCS7	нет	PKCS7	нулями
Длина шифртекста	кратна 8	$8 + \|X\|$	8 + кратное 8	$8 + 4 + \|X\|$
Зависимость блоков	нет	нет	есть	есть
Ошибка в бите портит	весь блок	один бит	бит и след. блок	имитовставка
Параллелится	полностью	полностью	только расшифрование	нет
Что защищает	секретность	секретность	секретность	целостность

6. Расхождения со стандартом и недочёты

Раздел обязателен к прочтению перед защитой: всё перечисленное – места, где преподаватель, сверивший код с текстом стандарта, может задать вопрос.

1. Первый блок гаммы вырабатывается с инкрементом

По п. 3.1 стандарта первый блок гаммы – это **неинкрементированный** результат шифрования синхропосылки: «Полученное в результате зашифрования заполнение N_1, N_2 образует первый 64-разрядный блок гаммы», и только для второго и дальнейших блоков счётчики увеличиваются. В коде инкремент выполняется **до** первого шифрования, поэтому поток гаммы сдвинут на один блок относительно стандарта.

Проверено на ключе `00..1F` и синхропосылке `0001020304050607` :

- по стандарту первый блок гаммы: `61A716F6245D1A0D` ;
- в коде: `9C7F2CC8C2927F84` .

На стойкость это не влияет (счётчики всё равно не повторяются, синхропосылка случайна), но шифртекст не совпадёт с другими реализациями ГОСТ.

2. Обратная связь использует счётчик, а не сам блок

По п. 4.1 стандарта гамма в режиме с обратной связью – это **прямо** зашифрованный предыдущий блок шифртекста: $\Gamma_i = F(Y_{i-1})$, то есть одно шифрование на блок. В коде вызывается `Gamma(prev, key, 8)` , то есть $\Gamma_i = F(n_3 \parallel n_4)$, где n_3, n_4 получены из `F(prev)` с инкрементом: два шифрования на блок и другая структура.

Проверено на том же ключе: Γ_2 по стандарту – `6ABE65A787CE7E56` , в коде – `D4CBC69C7B7D36DA` . По сути это не режим ГОСТ, а гаммирование по своему счётчику, где регистр обратной связи – шифр-

текст. Криптографически корректно (самосинхронизация работает, круговые проверки проходят), но с названием «режим ГОСТ 28147-89» не совпадает.

3. Опечатка в теоретической части отчёта
В разделе «Режим гаммирования» указано приращение `0x1010101B`. Правильные константы из приложения 2 стандарта: $C_1 =$ `0x01010104` и $C_2 =$ `0x01010101` – как в коде и в разделе «Реализация».

4. Нет привязки к эталонному шифртексту
Само издание ГОСТ 28147-89 не содержит примеров, и публичные тест-векторы привязаны к конкретным узлам замены. В работе проверка идёт сравнением с независимой реализацией **тех же уравнений** в тестах (`blockByStandard`, `macByStandard`). Это лучше, чем ничего, и ловит ошибки в порядке половин и в выборе битов имитовставки, но не ловит ошибку в самой функции f: если ошибочны и f, и её копия в тесте, тест пройдёт.

5. Мелкие недочёты реализации
- `unpad` не проверяет корректность дополнения: если последний байт не похож на длину дополнения, данные возвращаются как есть. Искажённый шифртекст может тихо дать обрезанный открытый текст.
- При несовпадении имитовставки вызывается `panic`, а не возвращается ошибка: проверка MAC смешана с преобразованием данных.
- Автоматических круговых тестов для четырёх режимов нет – есть только тест для блока; в отчёте это проверялось вручную через программу.

7. Реализация

7.1. Структура проекта

Файл	Содержимое
`internal/crypto/gost/gost.go`	узлы замены, расписание ключей, функция раунда, шифрование и расшифрование блока
`internal/crypto/gost/ecb.go`	режим простой замены
`internal/crypto/gost/gamma.go`	выработка гаммы и режим гаммирования
`internal/crypto/gost/gammaFeedback.go`	режим гаммирования с обратной связью
`internal/crypto/gost/mac.go`	выработка имитовставки и шифрование с MAC
`internal/crypto/gost/utils.go`	дополнение и снятие дополнения, дополнение нулями
`internal/crypto/gost/vectors_test.go`	восемь тестов по тексту стандарта
`internal/cipher/gost.go`	обёртка: четыре метода, вывод в виде hex-строки

Пакет `internal/crypto/gost` – чистые функции над `[]byte`, без ввода-вывода и знания о пользователе.

7.2. Формат шифртекста

Синхропосылка и имитовставка считаются параметрами протокола, но в программе передаются вместе с шифртекстом, каждый раз новые:

$$\text{гамма / обр. связь} = \underbrace{S}_{8\text{ байт}} \parallel \underbrace{\text{тело}}_{|X|\text{ байт}}$$

$$\text{гамма с MAC} = \underbrace{S}_{8\text{ байт}} \parallel \underbrace{\text{MAC}}_{4\text{ байта}} \parallel \underbrace{\text{тело}}_{|X|\text{ байт}}$$

Ввод и вывод – шестнадцатеричные строки, поэтому 8 байт синхропосылки занимают первые 16 символов, а имитовставка – следующие 8. У простой замены префиксов нет.

7.3. Методы

`Methods()` возвращает четыре строки: `Simple`, `Gamma`, `Gamma with feedback`, `Gamma with MAC`. Ключ проверяется на длину 32 байта. При расшифровании с имитовставкой проверяется, что вход длиннее 12 байт, иначе внутри `MAC` возникнет выход за границу.

8. Проверка работоспособности

Тесты запускаются командой `go test ./internal/crypto/gost/`. Всего восемь тестов, все проходят.

1. **Узлы замены.** Проверяется, что все восемь строк – перестановки чисел от 0 до 15, без повторов и без выходов за 15. Это ровно то требование, которое стандарт предъявляет к блоку подстановки.
2. **Расписание ключей для шифрования.** Сверяется с текстом п. 2.1: $X_0, \ldots, X_7$ трижды, затем $X_7, \ldots, X_0$.
3. **Расписание ключей для расшифрования.** Сверяется с п. 2.2: восемь прямо, дальше 24 в обратном циклическом порядке.
4. **Имитовставка и 16 циклов.** Проверяется, что расписание для 16 раундов совпадает с первыми 16 раундами полного расписания, то есть разворота на этом отрезке нет.
5. **Порядок половин блока.** Сравнивается с прямой реализацией уравнений п. 2.1, где последний цикл обработан отдельно. Это тест именно на тот тонкий момент с обменом ролей накопителей в 32-м цикле.
6. **Круговой тест блока.** `BlockDecrypt32(BlockEncrypt32(x))` возвращает исходный блок.
7. **Константы приращения счётчиков.** Первый блок гаммы сверяется с блоком, полученным приращением на C_1 и C_2 – то есть закрепляет **текущее** поведение кода.
8. **Имитовставка.** Для четырёх длин (8, 16, 24, 32 бита) сверяется с формулой п. 5.3: старшие l бит N_1, прижатые влево.

> **Чего в тестах нет**
> Нет круговых проверок для режимов и нет сверки с внешним эталонным шифртекстом – см. п. 4 и 5 раздела о расхождениях. Это первые кандидаты на доработку, если спросят, как обеспечивалась проверка.

9. Вопросы на защите

? Что делалось в работе?
Блочный шифр ГОСТ 28147-89: восемь узлов замены, функция раунда, 32 цикла, расписание ключей, шифрование и расшифрование блока. Четыре режима: простая замена, гаммирование, гаммирование с обратной связью, имитовставка.

? Какие параметры у шифра?
Блок 64 бита, ключ 256 бит, 32 цикла, подключ 32 бита, восемь узлов замены по 4 бита. Для имитовставки – 16 циклов.

? Почему стандарт не задаёт блок подстановки?
Он объявлен долговременным ключевым элементом: требование к нему одно – каждый из восьми узлов должен быть перестановкой чисел от 0 до 15. Набор берётся из параметра RFC 7836 `id-tc26-gost-28147-param-Z`. Поэтому «правильный ГОСТ» не один: с другим блоком подстановки шифртекст будет другим, и это не ошибка.

? Что такое сеть Фейстеля?
Блок делится на две половины. За цикл новая половина – это старая вторая, сложенная с результатом функции f от старой первой, а вторая половина становится старой первой. За счёт этого шифр обратим

при любой, даже необратимой функции f: для расшифрования половины просто возвращаются на места и f применяется повторно.

? Почему в 32-м цикле роли накопителей не меняются?

Так записано в стандарте: в последнем цикле результат из сумматора пишется в N_2, а N_1 сохраняет старое заполнение. В коде это отдельной веткой не выделено, потому что цикл с обменом половин ровно 32 раза даёт тот же результат.

? Как выглядит функция раунда?

Сложение входа с подключом по модулю 2^{32}, разбиение на восемь 4-битных частей начиная с младшей, замена каждой части своим узлом, склейка и циклический сдвиг влево на 11 бит.

? Почему сдвиг именно на 11 бит?

Число 11 задано стандартом и не совпадает с размером слова, поэтому сдвиг перемешивает биты между разрядами и обеспечивает диффузию без потери обратимости: f остаётся биекцией.

? Как формируется расписание ключей?

При шифровании восемь подключей повторяются трижды, затем идут в обратном порядке. При расшифровании первые восемь идут прямо, остальные 24 – циклически в обратном порядке. Для имитовставки берутся первые 16 циклов без разворота.

? Чем отличается алгоритм шифрования в последнем цикле?

В последнем цикле накопители не меняются местами: N_1 сохраняет старое значение, результат идёт в N_2. В остальных циклах наоборот: старое N_1 переписывается в N_2, а в N_1 записывается результат.

? Чем отличаются режимы между собой?

Простая замена шифрует блоки независимо, синхропосылки нет, нужна F^{-1}. Гаммирование складывает данные с потоком, вычисляемым из счётчиков, поток не зависит от данных, поэтому F^{-1} не нужна и режим параллелится полностью. Гаммирование с обратной связью берёт регистром предыдущий шифртекст, из-за чего зашифрование последовательно, а ошибка в одном бите портит следующий блок целиком. Имитовставка не шифрует ничего, а вычисляется на ключе и даёт целостность и подлинность.

? Зачем режиму гаммирования синхропосылка?

Чтобы расшифрование с самого начала знало, с чего начинать счётчики: Г строится из $F(S)$. Секретной она не является, её передают открыто вместе с шифртекстом. Повтор синхропосылки при том же ключе делает два сообщения зависимыми друг от друга, поэтому она берётся случайной.

? Почему счётчик n_4 растёт по модулю $2^{32}-1$, а не 2^{32}?

Так задано в стандарте. При сложении по модулю 2^{32} перенос терялся бы, и счётчик повторился бы раньше, чем накопятся 2^{32} значений. Значение `0xFFFFFFFF` при этом трактуется как ноль.

? Чем режим гаммирования отличается от гаммирования с обратной связью?

Источник потока. В гаммировании он строится из счётчиков, которые зависят только от ключа и синхропосылки: поток можно вычислить заранее, ошибка не размножается, есть произвольный доступ к блокам. В режиме с обратной связью поток зависит от предыдущего шифртекста: зашифрование строго последовательное, ошибка в бите портит следующий блок целиком, зато режим самосинхронизируется.

? Что такое самосинхронизация?

Состояние режима – последний принятый блок. Если блок потерян, искажён или вставлен лишний, через один блок регистр снова содержит верные данные и расшифрование продолжается правильно. Безвозвратно теряется один блок.

? Как устроена имитовставка?

Вычислительная схема на первых 16 циклах: каждый блок складывается с накопленным значением поразрядно по модулю 2, затем идёт шифрование 16 циклами, результат становится новым накопленным значением. Из него берутся первые l бит, где $1 \le l \le 32$.

? Почему имитовставка берёт старшие биты, а не младшие?

Так в стандарте: $I_l = [a_{32-l+1} \parallel \ldots \parallel a_{32}]$ – это l старших разрядов N_1, прижатых влево. В коде это `binary.BigEndian.PutUint32`, чтобы старшие биты оказались в начале байтовой строки.

? Чем дополнение для имитовставки отличается от дополнения в простой замене?

Для имитовставки последний блок дополняется **нулями**, во всех режимах кроме гаммирования – по

PKCS#7 значением длины. Разница принципиальная: дополнение нулями необратимо, и имитовставка не может быть вычислена без знания длины сообщения.

? Зачем в режиме с имитовставкой нужна синхропосылка и как всё устроено?

Режим составной: сообщение шифруется гаммированием, а имитовставка считается по **открытому** тексту. При расшифровании текст восстанавливается, имитовставка пересчитывается и сравнивается с переданной. В программе при несовпадении вызывается паника.

? Проверялось ли, что режимы совместимы с другими реализациями ГОСТ?

Здесь честный ответ такой: в работе сверка шла с независимой реализацией уравнений стандарта внутри тестов, а не с внешним эталонным шифртекстом. Кроме того, у режимов гаммирования есть расхождения с текстом стандарта: первый блок гаммы вырабатывается с инкрементом, а обратная связь реализована через счётчик, а не напрямую через $F(Y_{i-1})$. Для учебной работы это работоспособное решение, но с выходом на interoperability оно не годится.

? Какие расхождения со стандартом есть в работе?

Первое: по стандарту первый блок гаммы – неинкрементированный результат шифрования синхропосылки, в коде инкремент делается до первого шифрования. Второе: по стандарту гамма в режиме с обратной связью равна $F(Y_{i-1})$, то есть одно шифрование на блок, в коде используется счётчик, то есть два. Остальное – блочный шифр, расписание ключей и имитовставка — стандарту соответствует.

? Как проверялась корректность?

Восемь тестов по тексту стандарта: узлы замены являются перестановками, расписание ключей для шифрования и расшифрования совпадает с пп. 2.1 и 2.2, порядок половин блока – с уравнениями п. 2.1, имитовставка берёт старшие биты N_1 по п. 5.3. Плюс круговой тест блока. Круговые проверки режимов в автоматических тестах не закреплены.

? Что даёт имитовставка и чем это отличается от шифрования?

Шифрование даёт секретность, но не целостность: в режиме гаммирования с обратной связью противник может инвертировать выбранные биты шифртекста и предсказуемо изменить открытый текст, не зная ключа. Имитовставка вычисляется на ключе, поэтому получатель может проверить и целостность, и то, что отправитель знает ключ.

? Чем этот ГОСТ отличается от `belt` из работы №2?

Здесь сеть Фейстеля, блок 64 бита, 32 цикла, подстановка 4 бита в 4 бит и восемь узлов, которые выбираются отдельно, а расписание ключей с разворотом. Там схема подстановок-перестановок, блок 128 бит, 8 тактов, одна восьмибитная подстановка, заданная стандартом, и нулевое расписание ключей.

? Считается ли реализация пригодной для применения?

Как учебная – да: блочный шифр и имитовставка соответствуют стандарту, всё показано на тестах. Для реальных данных режимы в таком виде непригодны: шифрование не даёт целостности, а расхождения в гаммировании ломают совместимость.